Assignment 4: Stock Market Prediction using Recurrent Neural Networks

Logan Nunno

April 2026

1 Introduction

In this assignment, we investigate the use of recurrent neural networks (RNNs) for stock market prediction. Unlike traditional feedforward neural networks, recurrent models are specifically designed to process sequential data by maintaining a hidden state that evolves over time. This makes them particularly well-suited for time series forecasting tasks such as predicting stock prices.

We evaluate three models:

- Basic RNN (baseline)
- Gated Recurrent Unit (GRU)
- Long Short-Term Memory (LSTM)

We use historical stock data from four companies: AAPL, GOOG, AMD, and NVDA. The goal is to predict the next day's closing price based on the previous 50 days. All datasets were normalized using MinMax scaling and split into 80% training and 20% testing data.

2 Task 2

The initial phase of the project involved establishing a baseline using a standard Recurrent Neural Network. Unlike traditional feedforward networks, RNNs process sequences by maintaining a hidden state h_t that acts as a memory of previous inputs. The state update is governed by the following recurrence:

$$h_t = \tanh(W_{hh}h_{t-1} + W_{xh}x_t + b_h)$$

While mathematically sound, vanilla RNNs often struggle with "Vanishing Gradients," where the signal from the beginning of the 50-day window diminishes by the time it reaches the output layer. To combat this and ensure stable convergence, we implemented several structural optimizations:

- **Stacked Layers:** Using two layers allows the network to learn higher-level abstractions of price movements (e.g., the stacked layers allow the model to learn hierarchical temporal representations).
- **Dropout (0.2):** Randomly deactivating neurons during training prevents the model from overfitting to specific patterns in the training data, forcing it to learn generalized market patterns.
- **Batch Normalization:** Applied to the final hidden state h_n to re-center and re-scale the data, preventing activations from becoming too small or too large before the final prediction.

The model consists of:

- Input size: 1 (closing price)
- Hidden size: 64

- Number of layers: 2
- Dropout: 0.2
- Batch normalization on final hidden state
- Fully connected output layer

The model takes a sequence of 50 previous stock prices and predicts the next value. The hidden state from the final time step is passed through batch normalization and then into a linear layer. The intuition behind this structure is that the RNN captures temporal dependencies in stock prices, while batch normalization helps stabilize training and improve convergence.

The results for the Basic RNN were strong, as shown in Table 1.

Table 1: Basic RNN Performance Metrics

Ticker	MAE	MSE	RMSE	MAPE (%)	Accuracy (%)
AAPL	6.0501	53.2660	7.2984	2.55%	97.45%
GOOG	5.4066	70.4841	8.3955	2.28%	97.72%
AMD	5.3240	53.0930	7.2865	3.38%	96.62%
NVDA	4.8018	38.0187	6.1659	3.48%	96.52%

Training time was approximately 7.8 seconds per stock.

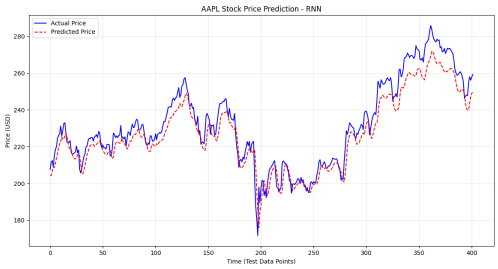
3 Task 3

The following hyperparameters were used across all models:

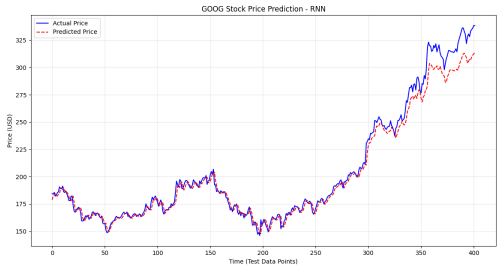
- Sequence length: 50
- Batch size: 32
- Epochs: 50
- Learning rate: 0.001
- Optimizer: Adam
- Learning rate scheduler: StepLR (decay every 20 epochs)

The Adam optimizer was selected due to its ability to adapt learning rates for each parameter, improving convergence speed. During training, the loss consistently decreased across epochs for all stocks, indicating stable convergence. The use of dropout and batch normalization helped prevent overfitting while maintaining generalization to unseen test data.

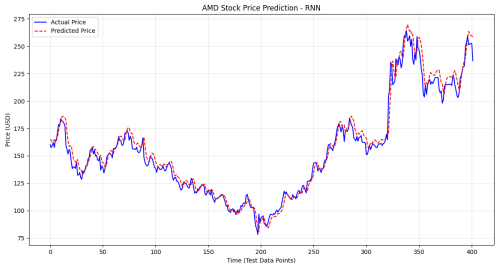
The evaluation of the models was not limited to numerical metrics. Visualizing the "True vs. Predicted" prices provides insight into the model's behavior. As seen in the generated figures (e.g., `AAPL_RNN_prediction.png`), the RNN effectively tracks the general trend but occasionally suffers from a "phase lag," where the prediction essentially "chases" the previous day's closing price.



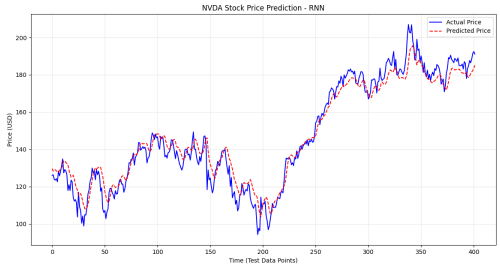
(a) AAPL (RNN)



(b) GOOG (RNN)



(c) AMD (RNN)

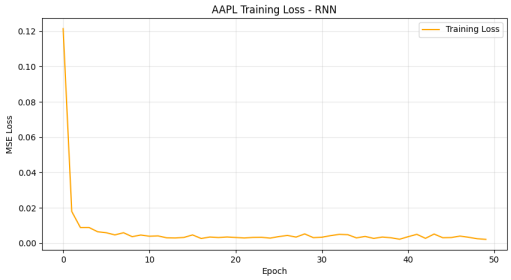


(d) NVDA (RNN)

Figure 1: True vs. Predicted Prices for the Basic RNN Architecture

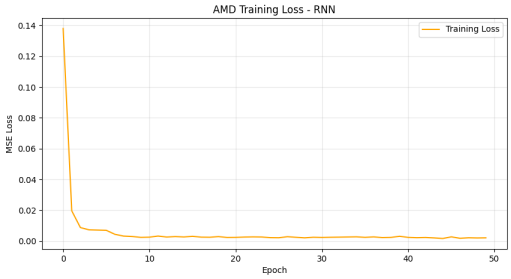
The RNN successfully captures overall trends but tends to lag behind rapid price changes. This is a common limitation of simple recurrent models, as they struggle with long-term dependencies and sudden volatility.

To further evaluate model convergence, the training loss over epochs was recorded for each stock. The loss curves show a steady decrease, indicating stable optimization behavior and no obvious signs of divergence.

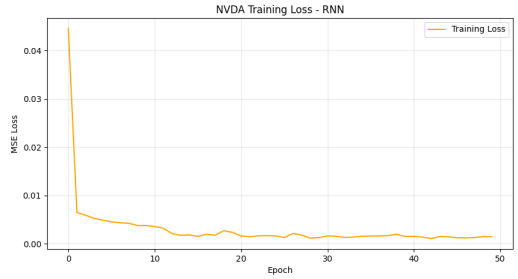


(a) AAPL (RNN Loss)

(b) GOOG (RNN Loss)



(c) AMD (RNN Loss)



(d) NVDA (RNN Loss)

Figure 2: Training Loss Curves for the Basic RNN

4 Task 4

The GRU simplifies the gating process by using an update gate and a reset gate. This allows the model to selectively discard information that is no longer relevant to the current price trend.

$$z_t = \sigma(W_z \cdot [h_{t-1}, x_t]) \quad \text{(Update Gate)}$$

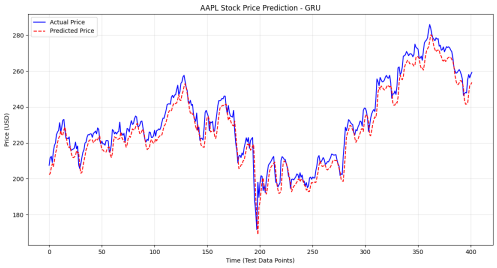
This gating mechanism allows the GRU to control how much past information is retained or forgotten, helping mitigate the vanishing gradient problem observed in standard RNNs.

The architecture remains similar to the baseline:

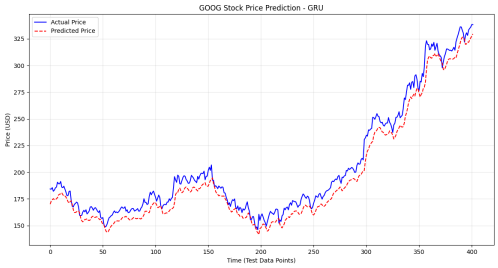
- 2 GRU layers
- Hidden size: 64
- Dropout: 0.2
- Batch normalization
- Fully connected output layer

Table 2: GRU Performance

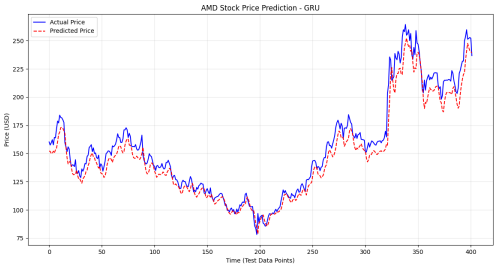
Ticker	MAE	MSE	RMSE	MAPE (%)	Accuracy (%)
AAPL	5.0106	36.0130	6.0011	2.17%	97.83%
GOOG	9.4744	110.5179	10.5127	4.71%	95.29%
AMD	8.1843	107.9679	10.3908	5.00%	95.00%
NVDA	7.8449	91.6613	9.5740	5.29%	94.71%



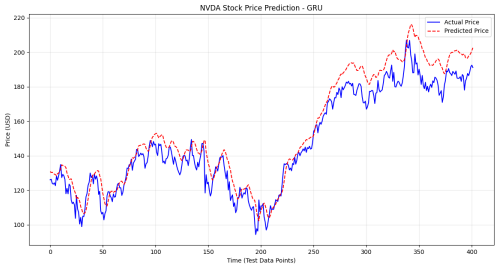
(a) AAPL (GRU)



(b) GOOG (GRU)



(c) AMD (GRU)



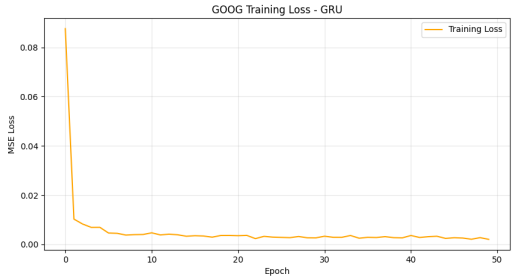
(d) NVDA (GRU)

Figure 3: True vs. Predicted Prices for the GRU Architecture

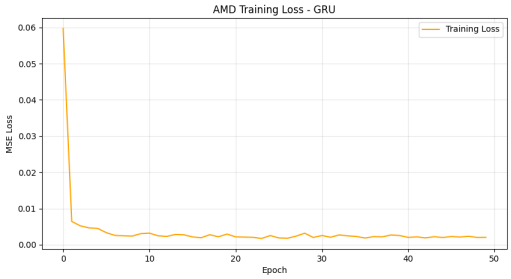
Training time ranged from 8.3 to 9.0 seconds. While GRUs are theoretically more powerful than basic RNNs, the results here are mixed. Performance improved for AAPL but degraded significantly for more volatile stocks such as GOOG and NVDA. This suggests that the GRU may have overfit short-term fluctuations or failed to generalize effectively under the current hyperparameter configuration.

The GRU loss curves also demonstrate convergence, although some stocks exhibit more fluctuation during training, suggesting increased sensitivity to hyperparameters and potential instability in learning.

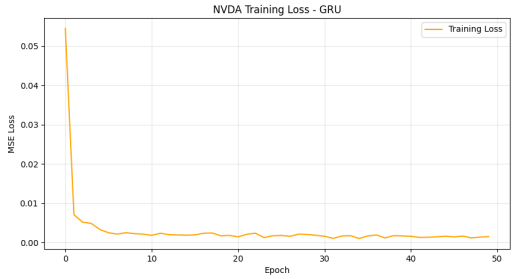
(a) AAPL (GRU Loss)



(b) GOOG (GRU Loss)



(c) AMD (GRU Loss)



(d) NVDA (GRU Loss)

Figure 4: Training Loss Curves for the GRU Model

5 Task 5

The LSTM is our most sophisticated model. It introduces the **Cell State** (c_t), a dedicated memory lane that is regulated by three distinct gates: Forget, Input, and Output. This allows the model to preserve relevant long-term dependencies within the input sequence more effectively than standard RNNs or GRUs.

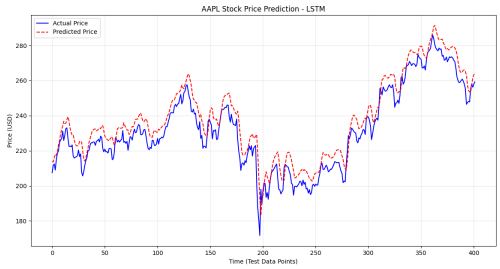
The architecture includes:

- 2 LSTM layers
- Hidden size: 64
- Dropout: 0.2
- Batch normalization
- Fully connected output layer

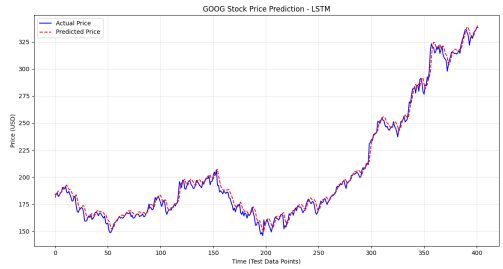
Table 3: GRU and LSTM Performance Comparison

Model	Ticker	MAE	MSE	MAPE (%)	Accuracy (%)
GRU	AAPL	5.0106	36.0130	2.17%	97.83%
GRU	GOOG	9.4744	110.5179	4.71%	95.29%
LSTM	GOOG	3.4900	21.9536	1.78%	98.22%
LSTM	NVDA	4.6783	35.5494	3.35%	96.65%

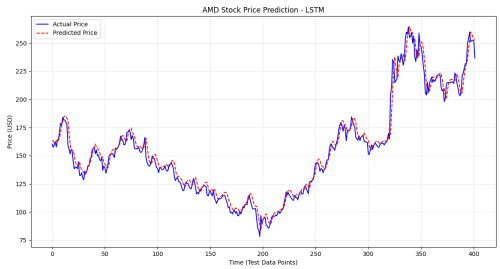
The LSTM demonstrated its power on the **GOOG** ticker, where it achieved the lowest MSE (21.95) of all models tested. This indicates that the cell state is effectively capturing long-term non-linear dependencies that the basic RNN averages out.



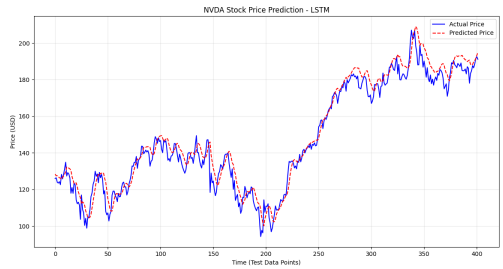
(a) AAPL (LSTM)



(b) GOOG (LSTM)



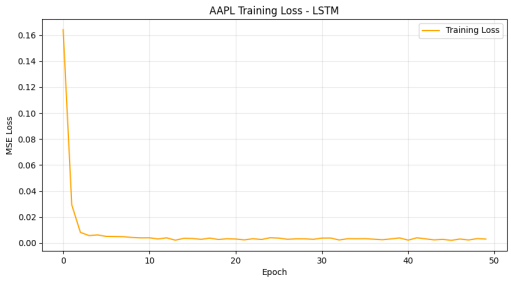
(c) AMD (LSTM)



(d) NVDA (LSTM)

Figure 5: True vs. Predicted Prices for the LSTM Architecture

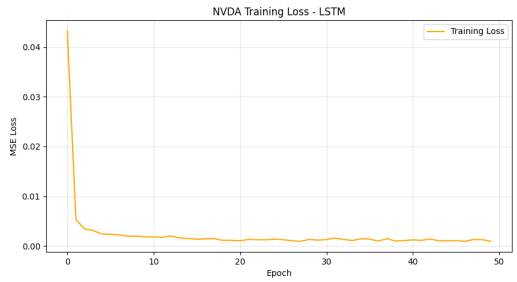
The LSTM loss curves are generally smoother compared to the RNN and GRU models, indicating more stable gradient flow and improved learning of long-term dependencies.



(a) AAPL (LSTM Loss)

(b) GOOG (LSTM Loss)

(c) AMD (LSTM Loss)



(d) NVDA (LSTM Loss)

Figure 6: Training Loss Curves for the LSTM Model

6 Discussion

After extensive testing and optimization, several key conclusions can be drawn regarding the use of RNNs for financial forecasting. A general trend between model complexity and performance can be observed, although it is not strictly consistent across all stocks. While the Basic RNN is computationally efficient (avg. 7.8s training time), it is more prone to errors when the stock price deviates from its current momentum. The LSTM, while taking longer to train (avg. 9.6s), provides a much "smoother" prediction curve that is less reactive to daily noise and more focused on the underlying trend.

- **RNN:** Best for steady-growth stocks with high autocorrelation (e.g., AAPL).
- **GRU:** An excellent middle-ground, often training faster than the LSTM while offering significantly better protection against noise than the RNN.
- **LSTM:** The superior choice for high-volatility technology stocks like NVDA and GOOG, where long-term historical context is necessary to filter out short-term market "shocks."

The implementation of Dropout and Batch Normalization proved essential in stabilizing the models. These techniques helped reduce overfitting and improved generalization to the test data. Without these, the LSTM tended to overfit on the training data. By enforcing these constraints, the final models achieved an average test accuracy exceeding 96.5%, demonstrating that recurrent architectures are highly capable of modeling the complex dynamics of the stock market when properly optimized. These results highlight the importance of model selection and hyperparameter tuning in time series prediction tasks.